

LAPORAN PRAKTIKUM 3
THREADS DAN CONCURRENCY



Disusun Oleh =
M. Ayatullah Mutawakki F (2410651011)

Dosen Pengampu =
Triawan Adi Cahyanto M.Kom

PROGRAM STUDY TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025

ISI LAPORAN

A. KALKULATOR PARALEL (TUGAS 1)

1. Dasar teori

Thread adalah unit eksekusi terkecil didalam proses. Satu proses dapat memiliki lebih dari satu thread yang bekerja secara bersamaan, berbagi memory dan sumber daya yang sama.

Dengan menggunakan thread ini program dapat mengerjakan beberapa tugas secara parallel, sehingga waktu eksekusi dapat menjadi lebih efisien.

Library pthread di C menyediakan fungsi-fungsi, seperti:

- Pthread_create() ini membuat thread
- Pthread_join() ini memnggu thread selesai
- Pthread_exit() mengakhiri thread

2. Kode kalkulator_paralel.c

```
#include <stdio.h>
#include <pthread.h>

float a, b;
float hasil_jumlah, hasil_kurang, hasil_kali, hasil_bagi;

// Fungsi untuk setiap operasi
void* tambah(void* arg) {
    hasil_jumlah = a + b;
    pthread_exit(NULL);
}

void* kurang(void* arg) {
    hasil_kurang = a - b;
    pthread_exit(NULL);
}

void* kali(void* arg) {
    hasil_kali = a * b;
    pthread_exit(NULL);
}

void* bagi(void* arg) {
    if (b != 0)
        hasil_bagi = a / b;
    else
        hasil_bagi = 0; // pembagian nol
    pthread_exit(NULL);
}
```

```

int main() {
    pthread_t t1, t2, t3, t4;

    printf("=== Program Kalkulator Paralel ===\n");
    printf("Masukkan angka pertama: ");
    scanf("%f", &a);
    printf("Masukkan angka kedua: ");
    scanf("%f", &b);

    // Membuat thread untuk tiap operasi
    pthread_create(&t1, NULL, tambah, NULL);
    pthread_create(&t2, NULL, kurang, NULL);
    pthread_create(&t3, NULL, kali, NULL);
    pthread_create(&t4, NULL, bagi, NULL);

    // Menunggu semua thread selesai
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);
    pthread_join(t4, NULL);

    // Menampilkan hasil
    printf("\n=== Hasil Perhitungan ===\n");
    printf("Penjumlahan: %.2f\n", hasil_jumlah);
    printf("Pengurangan: %.2f\n", hasil_kurang);
    printf("Perkalian : %.2f\n", hasil_kali);
    printf("Pembagian : %.2f\n", hasil_bagi);

    return 0;
}

```

3. Penjelasan kode

```

float a, b;
float hasil_jumlah, hasil_kurang, hasil_kali, hasil_bagi;

```

a dan b disini menyimpan input dari pengguna dan variabel hasil_jumlah dst. Itu untuk menyimpan hasil hitungan dari setiap thread.

```

// Fungsi untuk setiap operasi

```

Untuk fungsi-fungsinya setiap operasi baik itu (jumlah, kurang, kali, bagi), Masing-masing bertipe void agar bisa dijalankan oleh pthread.

pthread_exit(NULL); ini bertanda thread sudah selesai dijalankan.

Dan didalam operasi pembagian ada kode if (b != 0) ini untuk mencegah pembagian dengan nol.

Bagian main()

```
int main() {  
    pthread_t t1, t2, t3, t4;  
  
    printf("=== Program Kalkulator Paralel ===\n");  
    printf("Masukkan angka pertama: ");  
    scanf("%f", &a);  
    printf("Masukkan angka kedua: ");  
    scanf("%f", &b);
```

Untuk menampilkan judul program dan meminta input 2 angka dari pengguna menggunakan scanf.

```
// Membuat thread untuk tiap operasi  
pthread_create(&t1, NULL, tambah, NULL);  
pthread_create(&t2, NULL, kurang, NULL);  
pthread_create(&t3, NULL, kali, NULL);  
pthread_create(&t4, NULL, bagi, NULL);
```

selanjutnya membuat 4 thread untuk masing-masing operasi.

```
pthread_join(t1, NULL);  
pthread_join(t2, NULL);  
pthread_join(t3, NULL);  
pthread_join(t4, NULL);
```

Ini untuk menunggu operasi hitungan selesai sebelum menampilkan hasil akhirnya.

```
// Menampilkan hasil  
printf("\n=== Hasil Perhitungan ===\n");  
printf("Penjumlahan: %.2f\n", hasil_jumlah);  
printf("Pengurangan: %.2f\n", hasil_kurang);  
printf("Perkalian : %.2f\n", hasil_kali);  
printf("Pembagian : %.2f\n", hasil_bagi);
```

Menampilkan hasil setiap operasi.

4. Hasil outputnya

```
aqil@DESKTOP-RGGJ3FG:~$ gcc kalkulator_paralel.c -o kalkulator_paralel
aqil@DESKTOP-RGGJ3FG:~$ ./kalkulator_paralel
=== Program Kalkulator Paralel ===
Masukkan angka pertama: 10
Masukkan angka kedua: 2

=== Hasil Perhitungan ===
Penjumlahan: 12.00
Pengurangan: 8.00
Perkalian : 20.00
Pembagian : 5.00
```

5. Analisis

Keempat operasi ini dijalankan secara parallel.

Konsep ini sistem operasi yang mampu menjalankan banyak thread dalam satu proses untuk efisiensi waktu.

Walaupun perbedaannya tidak terlihat signifikan di program sederhana, pada skala besar (seperti server atau pemrosesan data besar), penggunaan thread sangat membantu meningkatkan performa.

Fungsi pthread_join agar hasil tidak ditampilkan sebelum semua perhitungan selesai.

Pengecekan if (b != 0) pada fungsi bagi mencegah error runtime (division by zero).

B. FILE PROCESSING (TUGAS 2)

1. Dasar teori

File processing adalah proses membaca, menulis, atau memanipulasi data yang tersimpan dalam file.

Thread memungkinkan program menjalankan beberapa bagian kode secara bersamaan. Dengan membagi pekerjaan (seperti menghitung kata di file besar) ke beberapa thread, program bisa memanfaatkan CPU multi-core untuk mempercepat waktu eksekusi.

Fungsi yang digunakan:

- `fopen()`: membuka file.
- `fread()` atau `fgets()`: membaca isi file.
- `pthread_create()` dan `pthread_join()`: membuat serta menunggu thread selesai.
- `clock()`: mengukur waktu eksekusi program.

2. Kode file_processing.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <time.h>

#define MAX_SIZE 1000000 // ukuran maksimum file teks

char buffer[MAX_SIZE];
int total_words = 0;

// Fungsi menghitung kata pada potongan teks
void* count_words(void* arg) {
    char* text = (char*)arg;
    int count = 0, in_word = 0;

    for (int i = 0; text[i] != '\0'; i++) {
        if (text[i] == ' ' || text[i] == '\n' || text[i] == '\t')
```

```

        in_word = 0;
    else if (in_word == 0) {
        in_word = 1;
        count++;
    }
}

int* result = malloc(sizeof(int));
*result = count;
pthread_exit(result);
}

// Fungsi versi single-thread
int count_words_single(char* text) {
    int count = 0, in_word = 0;
    for (int i = 0; text[i] != '\0'; i++) {
        if (text[i] == ' ' || text[i] == '\n' || text[i] == '\t')
            in_word = 0;
        else if (in_word == 0) {
            in_word = 1;
            count++;
        }
    }
    return count;
}

int main() {
    FILE* file = fopen("input.txt", "r");
    if (!file) {
        printf("Gagal membuka file!\n");
        return 1;
    }
}

```

```

fread(buffer, sizeof(char), MAX_SIZE, file);
fclose(file);

int length = strlen(buffer);
int mid = length / 2;

// --- Hitung dengan single-thread ---
clock_t start_single = clock();
int count_single = count_words_single(buffer);
clock_t end_single = clock();
double time_single = (double)(end_single - start_single) /
CLOCKS_PER_SEC;

// --- Hitung dengan dua thread ---
pthread_t t1, t2;
char* part1 = buffer;
char* part2 = buffer + mid;

// Pastikan part2 mulai di awal kata (bukan di tengah kata)
while (*part2 != '\0' && *part2 != ' ' && *part2 != '\n' && *part2
!= '\t') {
    part2++;
}

clock_t start_multi = clock();

int *res1, *res2;
pthread_create(&t1, NULL, count_words, (void*)part1);
pthread_create(&t2, NULL, count_words, (void*)part2);

pthread_join(t1, (void**)&res1);
pthread_join(t2, (void**)&res2);

```



```

        total_words = *res1 + *res2;
        clock_t end_multi = clock();
        double time_multi = (double)(end_multi - start_multi) /
        CLOCKS_PER_SEC;

        // --- Hasil ---
        printf("=== HASIL PERHITUNGAN ===\n");
        printf("Single-thread : %d kata (%.6f detik)\n", count_single,
        time_single);
        printf("Multi-thread   : %d kata (%.6f detik)\n", total_words,
        time_multi);

        free(res1);
        free(res2);
        return 0;
    }

```

3. Penjelasan kode

```

#include <pthread.h>
#include <time.h>
#define MAX_SIZE 1000000
char buffer[MAX_SIZE];
int total_words = 0;

```

Char buffer untuk menampung semua isi teks dari file.

Int total_word menyimpan total jumlah kata hasil gabungan dua thread.

```

void* count_words(void* arg) {
    char* text = (char*)arg;
    int count = 0, in_word = 0;
    ...
}

```

Fungsi ini dijalankan oleh setiap thread untuk menghitung jumlah kata pada setiap teks, yakni fungsi ini adalah pekerja untuk setiap thread dalam hitungan paralel.

```
int count_words_single(char* text) {  
    int count = 0, in_word = 0;  
    .....  
}
```

Count word single() ini sama seperti count word(), akan tetapi dipanggil langsung dari main() tanpa thread. Fungsi ini digunakan untuk membandingkan waktu eksekusi antara mode single dan multithread. Berarti ini disebut non paralel.

```
FILE* file = fopen("input.txt", "r");  
fread(buffer, sizeof(char), MAX_SIZE, file);  
fclose(file);  
int mid = strlen(buffer) / 2;  
char* part1 = buffer;  
char* part2 = buffer + mid;
```

Ini untuk membuka file teks dan seluruh isi dibaca ke dalam buffer.

Dan teks dibagi 2 agar bisa diproses oleh dua thread.

Dan

```
while (*part2 != '\0' && *part2 != ' ' && *part2 != '\n' && *part2 !=  
'\t') {  
    part2++;  
}
```

Ini untuk tidak terjadi pemotongan ditengah kata, agar hasil tetap akurat.

```
clock_t start_single = clock();  
int count_single = count_words_single(buffer);  
clock_t end_single = clock();  
double time_single = (double)(end_single - start_single) /  
CLOCKS_PER_SEC;
```

Mengukur waktu eksekusi dari single thread.

Dan untuk fungsi clock() mengambil waktu sebelum dan sesudah proses dan dihitung selisihnya.

Bagian multithread:

```
clock_t start_multi = clock();  
pthread_create(&t1, NULL, count_words, (void*)part1);  
pthread_create(&t2, NULL, count_words, (void*)part2);  
pthread_join(t1, (void**)&res1);  
pthread_join(t2, (void**)&res2);  
clock_t end_multi = clock();
```

Dua thread ini dibuat agar bisa menghitung kata secara bersamaan.

Fungsi dari pthread join() supaya memastikan program menunggu hingga kedua thread selesai.

Setelah selesai hasil digabung menggunakan kode ini:

```
total_words = *res1 + *res2;
```

```
printf("Single-thread : %d kata (%.6f detik)\n", count_single,  
time_single);  
printf("Multi-thread : %d kata (%.6f detik)\n", total_words, time_multi);
```

Disini kita tampilkan hasil perbandingan jumlah kata dan lama waktu eksekusi.

4. Hasil outputnya

Sebelum mengerjakan buat file teks, terserah apa saja bisa langsung buat di terminal.

seperti:

```
aqil@DESKTOP-RGGJ3FG:~$ nano input.txt
```

Lalu isi apa saja seperti ini:

```
[1/2]  
halo nama saya aqil  
saya dari Bondowoso  
saya kuliah di Universitas Muhammadiyah Jember  
Program Study Teknik Informatika
```

Lalu simpan ctrl O, enter, ctrl X lalu jalankan maka hasil outputnya:

```
aqil@DESKTOP-RGGJ3FG:~$ ./file_processing
=== HASIL PERHITUNGAN ===
Single-thread : 17 kata (0.000004 detik)
Multi-thread  : 23 kata (0.001049 detik)
```

5. Analisis hasil

Hasil perhitungan katanya tidak sama, padahal jumlah kata yang dimasukkan sama. Perbedaan ini terjadi karena pembagian file menjadi 2 bagian mungkin memotong ditengah sebuah kata, sehingga kata tersebut dihitung 2 kali oleh kedua thread.

Seperti dalam kata aqil:

- Thread 1 menghitung a sebagai satu kata.
- Dan thread 2 menghitung qil sebagai kata lain.

Perbedaan ini disebabkan oleh pembagian buffer yang tidak tepat di batas kata.

Nah untuk kecepatannya kenapa berbeda? Dikarenakan kalau pada file kecil threading membuat proses lebih lambat, tapi pada file besar, multi thread lebih unggul.

6. Dokumentasi Error

```
aqil@DESKTOP-RGGJ3FG:~$ gcc file_processing.c -o file_processing -pthread
./file_processing
Gagal membuka file!
```

Di sini gagal membuka file dikarenakan file teks yang ingin dibaca belum ada.

Caranya: buata file teksnya di terminal:

nano input.txt

lalu tulis beberapa kalimat didalamnya:

```
[1/2]
halo nama saya aqil
saya dari Bondowoso
saya kuliah di Universitas Muhammadiyah Jember
Program Study Teknik Informatika
```

Simpan file:

Tekan ctrl O lalu enter

Dan tekan ctrl X

Jalankan ulang programnya:

./file_processing

C. PRODUCER – CONSUMER (TUGAS 3)

1. Dasar teori

Produser consumer adalah masalah klasik dalam pemrograman parallel yang melibatkan sinkronasi antar proses yang memproduksi dan mengkonsumsi data dari buffer terbatas.

Masalah produser consumer ada 2 jenis proses:

- a. Producer: proses yang menghasilkan data dan menempatkan didalam buffer.
- b. Consumer: proses yang mengambil data atau item dari buffer untuk diproses.

Buffer memiliki kapasitas terbatas yang artinya hanya bisa menampung sejumlah item tertentu pada satu waktu.

Tantangan dalam masalah ini:

- a. Buffer penuh: producer tidak boleh menambah item ke buffer lagi jika sudah penuh.
- b. Buffer kosong: konsumen tidak boleh mengambil item dari buffer jika tidak ada item yang tersisa lagi.
- c. Eksekusi bersama: akses ke buffer harus dikelola agar tidak terjadi konflik antara producer dan konsumen. Hanya satu proses yang boleh mengakses buffer pada satu waktu.

Solusi masalah producer-consumer ini:

1. Semaphore: digunakan untuk mengelola akses ke buffer. Umumnya dua semaphores digunakan:
 - Full: menghitung jumlah item yang ada didalam buffer.
 - Empty: menghitung jumlah tempat kosong dalam buffer.
2. Mutex: semaphore biner 0 atau 1 yang memastikan bahwa hanya satu proses yang dapat mengakses buffer pada satu waktu, mencegah terjadinya race condition (balapan kondisi).

2. Kode producer_consumer.c

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define BUFFER_SIZE 10 // ukuran maksimum buffer
#define PRODUCE_COUNT 20 // total item yang akan diproduksi

int buffer[BUFFER_SIZE]; // array sebagai buffer
int count = 0; // jumlah item saat ini
int in = 0, out = 0; // posisi indeks producer dan consumer

pthread_mutex_t mutex;
pthread_cond_t not_full;
pthread_cond_t not_empty;

// ===== PRODUCER
=====

void* producer(void* arg) {
    for (int i = 0; i < PRODUCE_COUNT; i++) {
        int item = rand() % 100; // angka acak 0-99

        pthread_mutex_lock(&mutex); // kunci akses buffer
        while (count == BUFFER_SIZE) // jika penuh, tunggu
            pthread_cond_wait(&not_full, &mutex);

        // masukkan item ke buffer
        buffer[in] = item;
```

```

        in = (in + 1) % BUFFER_SIZE;
        count++;

        printf("Producer: menghasilkan %d | Jumlah buffer: %d\n", item,
count);

        pthread_cond_signal(&not_empty); // beri sinyal ke consumer
        pthread_mutex_unlock(&mutex); // buka kunci
        sleep(1); // delay agar output terbaca jelas
    }
    pthread_exit(NULL);
}

// ===== CONSUMER
=====

void* consumer(void* arg) {
    for (int i = 0; i < PRODUCE_COUNT; i++) {
        pthread_mutex_lock(&mutex); // kunci akses buffer
        while (count == 0) // jika kosong, tunggu
            pthread_cond_wait(&not_empty, &mutex);

        int item = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        count--;

        printf("Consumer: memproses %d | Sisa buffer: %d\n", item,
count);

        pthread_cond_signal(&not_full); // beri sinyal ke producer
        pthread_mutex_unlock(&mutex);
        sleep(2); // delay agar simulasi terlihat bergantian
    }
    pthread_exit(NULL);
}

```

```

    }

//      ===== MAIN      PROGRAM
=====

int main() {
    srand(time(NULL)); // seed random number

    pthread_t prod, cons;
    pthread_mutex_init(&mutex, NULL);
    pthread_cond_init(&not_full, NULL);
    pthread_cond_init(&not_empty, NULL);

    // buat thread producer dan consumer
    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    // tunggu kedua thread selesai
    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&not_full);
    pthread_cond_destroy(&not_empty);

    printf("\n=== Program selesai ===\n");
    return 0;
}

```

3. Penjelasan kode

```

    int buffer[BUFFER_SIZE];
    int count = 0;
    int in = 0, out = 0;

```



```
int buffer[BUFFER_SIZE];
```

Array disini untuk menyimpan item sementara

Dan count, in, dan out ini melacak banyak item, posisi indeks dimana produser akan menaruh data berikutnya, dan posisi indeks dimana consumer mengambil data berikutnya.

```
pthread_mutex_t mutex;  
pthread_cond_t not_full, not_empty;
```

Mutex ini memastikan tidak ada dua thread mengakses buffer bersamaan.

Not full memberi tau ke producer bahwa buffer sudah ada ruang kosong.

Dan not empty memberi sinyal ke consumer bahwa buffer sudah ada isi.

```
int item = rand() % 100;  
pthread_mutex_lock(&mutex);
```

Fungsi producer membuat angka acak dan mengunci mutex agar tidak nabrak consumer.

```
while (count == BUFFER_SIZE)  
pthread_cond_wait(&not_full, &mutex);
```

Kalau buffer penuh producer berhenti dan menunggu pemberitahuan bahwa buffer telah dikosongkan.

```
buffer[in] = item;  
in = (in + 1) % BUFFER_SIZE;  
count++;
```

Menambah data ke buffer menggunakan sistem indeks melingkar.

```
pthread_cond_signal(&not_empty);  
pthread_mutex_unlock(&mutex);
```

Memberi tahu consumer bahwa buffer sudah berisi. Dan membuka kunci mutex.

```
pthread_mutex_lock(&mutex);  
while (count == 0)  
pthread_cond_wait(&not_empty, &mutex);
```

Consumer menunggu sampai ada data di buffer.

```
int item = buffer[out];  
out = (out + 1) % BUFFER_SIZE;  
count--;
```

Consumer mengambil data, memindahkan indeks ke posisi berikutnya, dan mengurangi count.

```
pthread_cond_signal(&not_full);  
pthread_mutex_unlock(&mutex);
```

Memberi sinyal ke producer bahwa buffer sudah ada ruang kosong.

```
pthread_create(&prod, NULL, producer, NULL);  
pthread_create(&cons, NULL, consumer, NULL);
```

Membuat 2 thread secara paralel.

Pthread join ini memastikan agar program utama menunggu sampai kedua thread selesai.

```
pthread_mutex_destroy(&mutex);  
pthread_cond_destroy(&not_full);  
pthread_cond_destroy(&not_empty);
```

Memberishkan resource setelah program selesai.

4. Hasil outputnya

```
aqil@DESKTOP-RGGJ3FG:~$ gcc producer_consumer.c -o producer_consumer -pthread
./producer_consumer
Producer: menghasilkan 99 | Jumlah buffer: 1
Consumer: memproses 99 | Sisa buffer: 0
Producer: menghasilkan 31 | Jumlah buffer: 1
Consumer: memproses 31 | Sisa buffer: 0
Producer: menghasilkan 18 | Jumlah buffer: 1
Producer: menghasilkan 23 | Jumlah buffer: 2
Producer: menghasilkan 38 | Jumlah buffer: 3
Consumer: memproses 18 | Sisa buffer: 2
Producer: menghasilkan 59 | Jumlah buffer: 3
Consumer: memproses 23 | Sisa buffer: 2
Producer: menghasilkan 26 | Jumlah buffer: 3
Producer: menghasilkan 91 | Jumlah buffer: 4
Consumer: memproses 38 | Sisa buffer: 3
Producer: menghasilkan 6 | Jumlah buffer: 4
Producer: menghasilkan 55 | Jumlah buffer: 5
Consumer: memproses 59 | Sisa buffer: 4
Producer: menghasilkan 0 | Jumlah buffer: 5
Producer: menghasilkan 65 | Jumlah buffer: 6
Consumer: memproses 26 | Sisa buffer: 5
Producer: menghasilkan 26 | Jumlah buffer: 6
Producer: menghasilkan 98 | Jumlah buffer: 7
Consumer: memproses 91 | Sisa buffer: 6
Producer: menghasilkan 28 | Jumlah buffer: 7
Producer: menghasilkan 90 | Jumlah buffer: 8
Consumer: memproses 6 | Sisa buffer: 7
Producer: menghasilkan 10 | Jumlah buffer: 8
Producer: menghasilkan 34 | Jumlah buffer: 9
Consumer: memproses 55 | Sisa buffer: 8
Producer: menghasilkan 70 | Jumlah buffer: 9
Producer: menghasilkan 43 | Jumlah buffer: 10
Consumer: memproses 0 | Sisa buffer: 9
Consumer: memproses 65 | Sisa buffer: 8
Consumer: memproses 26 | Sisa buffer: 7
Consumer: memproses 98 | Sisa buffer: 6
Consumer: memproses 28 | Sisa buffer: 5
Consumer: memproses 90 | Sisa buffer: 4
Consumer: memproses 10 | Sisa buffer: 3
Consumer: memproses 34 | Sisa buffer: 2
Consumer: memproses 70 | Sisa buffer: 1
Consumer: memproses 43 | Sisa buffer: 0

=== Program selesai ===
```

5. Analisis hasil

Producer dan konsumernya berjalan secara bergantian, producer menunggu saat buffer penuh dan consumer menunggu saat buffer sedang kosong, dan tidak ada terjadi data race karena akses buffer dijaga mutex.

KESIMPULAN

Cara kerja multithreading dengan cara menjalankan empat operasi (tambah, kurang, bagi, kali) secara bersamaan menggunakan 4 thread.

Program perbandingan penghitungan kata single thread dan multi thread. Yang mana hasilnya multi thread kadang kurang akurat jika pembagian teksnya tidak tepat dan lebih lambat jika filenya kecil.

Program menerapkan konsep sinkronisasi antar thread menggunakan mutex. Producer menghasilkan data dan consumer memprosesnya dengan aturan antrian buffer